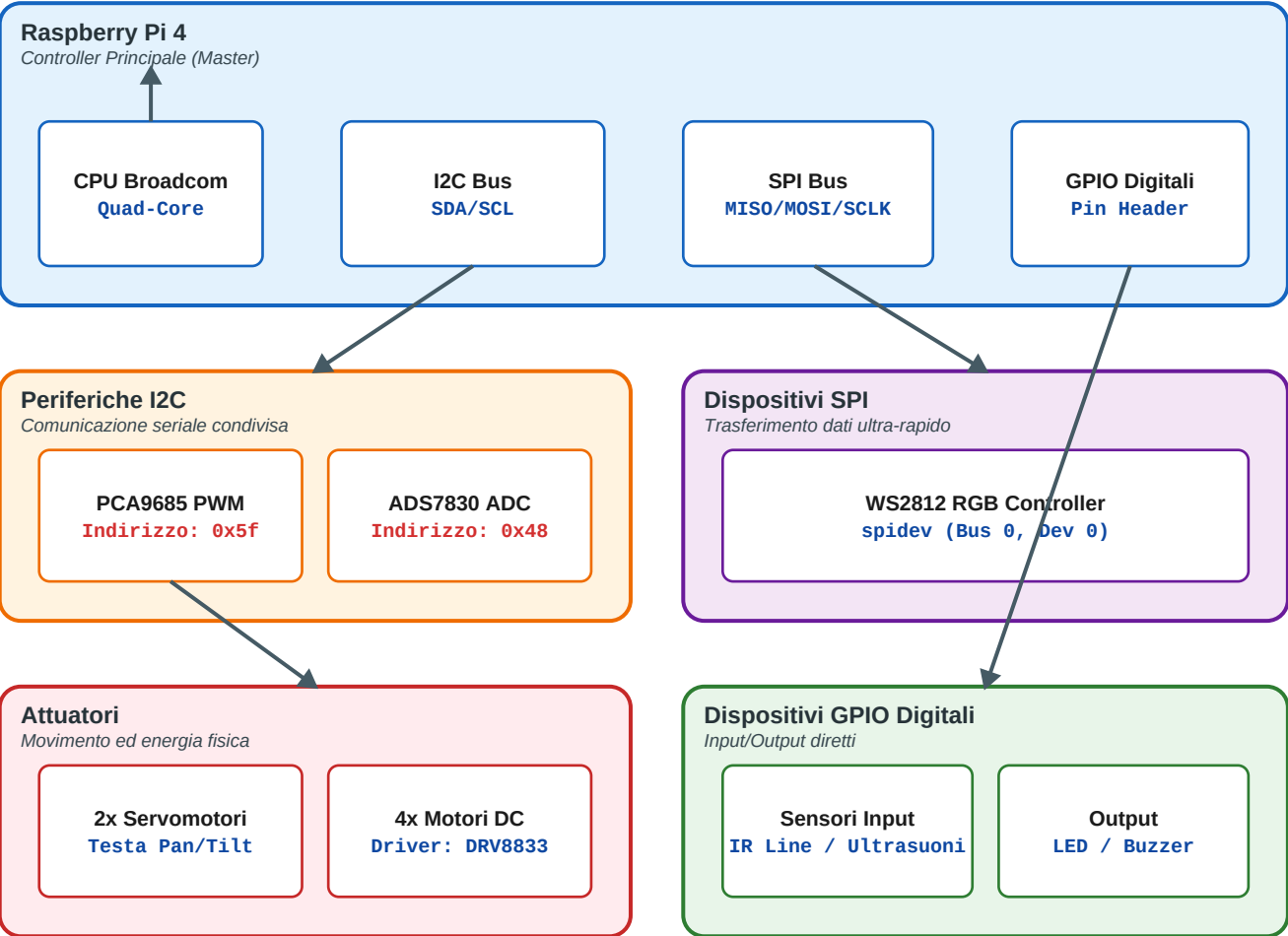


Adept 4WD Smart Car - Architettura Hardware Mock

Guida e analisi della simulazione hardware locale su macOS

Schema di Collegamento dell'Hardware Simulato



1. Bus I2C e Mappatura Pin (board.py, busio.py)

Il bus **I2C (Inter-Integrated Circuit)** consente a più circuiti integrati di comunicare su una linea condivisa usando solo due segnali: **SCL (Serial Clock)**, che fornisce gli impulsi temporali sincroni generati dal Master (Raspberry Pi), e **SDA (Serial Data)**, che trasporta i dati binari bidirezionali. Ciascun dispositivo slave possiede un indirizzo esadecimale univoco (ad esempio 0x5f per il controller dei motori e 0x48 per il convertitore ADC).

La Simulazione Mock:

- **board.py**: Definisce le costanti stringa fittizie per SDA e SCL.
- **busio.py**: Simula l'inizializzazione del bus I2C. Supporta il protocollo di Context Manager (`__enter__` / `__exit__`), consentendone l'uso sicuro con l'istruzione 'with' di Python.

2. Controllo degli Attuatori PWM (adafruit_pca9685.py, adafruit_motor/)

Nel robot fisico, il Raspberry Pi non pilota i motori o i servo direttamente. Invece, invia comandi via I2C al chip **PCA9685**, un controller PWM hardware dedicato a 16 canali. I servomotori ruotano su un angolo specifico (0-180 gradi) in base alla larghezza dell'impulso ricevuto, mentre i motori a corrente continua sono pilotati da segnali PWM passati attraverso un driver a ponte-H (**DRV8833**) che modula la velocità (throttle) e la direzione di rotazione.

La Simulazione Mock:

- **adafruit_pca9685.py**: Crea il chip simulato esponendo un array di 16 canali fittizi.
- **adafruit_motor/servo.py**: Permette di assegnare un angolo e stampa un avviso di WARNING se si superano i limiti fisici (0-180°).
- **adafruit_motor/motor.py**: Gestisce la proprietà 'throttle' (da -1.0 a 1.0) registrando in console la percentuale di spinta e la direzione.

3. GPIO e Comunicazione SPI (gpiozero.py, spidev.py)

I pin **GPIO (General Purpose Input/Output)** sono i piedini esposti del Raspberry Pi configurati come ingressi o uscite digitali: le uscite (LED, Buzzer) sono attivate a livello alto o basso; gli ingressi leggono i sensori di tracciamento linea (0 per il nero e 1 per il bianco). Il sensore ad ultrasuoni **HC-SR04** misura il tempo che intercorre tra l'invio dell'onda (Trigger) e il suo ritorno (Echo) per calcolare la distanza di un ostacolo. I LED posteriori **WS2812** richiedono velocità ultra-rapide e sono pilotati dal bus seriale **SPI** via spidev.

La Simulazione Mock:

- **gpiozero.py**: Fornisce classi mirate (LED, TonalBuzzer, InputDevice, DistanceSensor) che consentono di cambiare o simulare stati logici e distanze (es. per simulare l'ostacolo durante lo sviluppo).
- **spidev.py**: Emula il modulo del kernel Linux offrendo la classe SpiDev e il metodo xfer2, evitando crash di importazione su macOS.

Tabella degli Indirizzi I2C del Robot

Componente	Indirizzo I2C	Ruolo nel Robot
PCA9685	0x5f	Controller PWM per servomotori e motori DC (ruote).
ADS7830	0x48	Convertitore ADC per fotoresistenze e voltaggio batteria.
SSD1306	0x3c	Display OLED di stato per mostrare IP e messaggi.